

1 Motivation

An operating system can only be a toy unless it can self-host, that is, run without the help of another other OS, and able to self-evolve by compiling and replacing its own kernel.

So far, labs we have done in CS140E provides solid “proof of concept” steps for various OS components, but it will take significant effort to put them together into a self-hosting OS with both a kernel that allows interaction between user and programs, as well as a user-space toolchain which can support development of both the kernel and user programs.

This project aims to build such a self-hosting OS by combining and developing the components that we built across different labs, which includes the following features on top of what we already have from labs:

- A two-level page table implementation to allocate 4KB pages for user programs.
- A POSIX-compatible syscall interface, allowing execution of binaries ported from Linux.
- An ELF loader (static only) to load Linux-compatible user programs.
- A FAT32 implementation with interface to write, create, and unlink files.
- A Linux-like file descriptor interface to allow user programs to interact with files and devices in a unified way.
- A kuser interface ported from Linux, to allow libc programs to run without modification.
- A tty terminal driver over HDMI, with basic ANSI support to allow direct interaction with the OS without needing a serial console.
- A simi-vibe-coded USB driver to allow USB keyboards to be used for input.
- A vibe-patched TCC toolchain to allow compilation of both kernel and user programs on the pi.

2 Challenges

The most challenging part of this project is the USB driver, since the DCW controller was not well documented. The communication between the pi and the controller were mainly reverse engineered from the uspi project by Claude. Additionally, the USB protocol for HID devices also vary greatly across different devices with different connection speed, which mainly involves a lot of trial and error and tracing on Linux to figure out the correct sequence of commands to send to the controller to get the keyboard working. For more details on the USB driver, please refer to the writeup in <https://sites.tianleyu.com/unic/cs140e/usb.md>.

Surprisingly, the FAT32 and file descriptor implementation were the second most challenging part. Due to my early decision on the interfaces of the file system, it was not at all compatible with what Linux-style user programs expect, leading to both the reading and writing parts of the file system to be fully refactored multiple times to optimize stability and performance.